Hylo

Security Assessment

May 5th, 2025 — Prepared by OtterSec

Thibault Marboud thibault@osec.io

Nicola Vella nick0ve@osec.io

Robert Chen r@osec.io

Table of Contents

01 — Executive Summary

Overview

Hylo engaged OtterSec to assess the `stability-pool` and `exchange` programs. This assessment was conducted between February 10th and February 27th, 2025. We conducted a follow-up audit between April 2nd and April 30th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 7 findings throughout this audit engagement.

In particular, we found two **critical** vulnerabilities. The first is caused by missing validation of the `remaining_accounts` array when loading the LST registry. This could allow an attacker to omit or duplicate LST blocks, which would let them manipulate the collateral ratio (OS-HYL-ADV-00). The second issue is related to the protocol's use of exponentially weighted moving average (EMA) prices during minting. This could allow an attacker to take advantage of outdated pricing data to mint tokens at unfair rates (OS-HYL-ADV-01).

We also found a **medium-severity** issue where an attacker could bypass mint fees for `xSOL` (OS-HYL-ADV-02). In addition, we reported two **low-severity** issues, suggesting improvements to Pyth's `PriceData` oracle validation (OS-HYL-ADV-03) and removal of unnecessary zero-value operations (OS-HYL-ADV-04).

We also made some recommendations to improve the protocol. These include adding custom error messages to make debugging easier and improve safety (OS-HYL-SUG-00), and adding slippage checks during redemption, minting, and swapping to protect users from sudden price changes (OS-HYL-SUG-01).

We appreciate the client's cooperation and quick response in addressing the issues we found during the audit.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/hylo-so/protocol. This audit was performed against commit 3c91e4a. We further reviewed the codebase in a follow-up audit against 796267d.

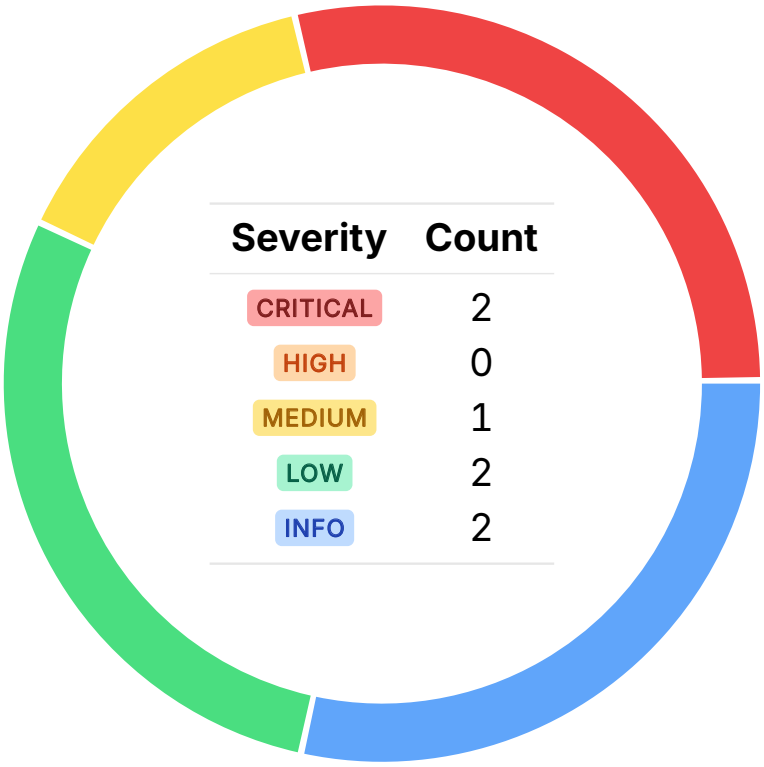
Brief descriptions of the programs are as follows:

Name	Description
exchange	A DeFi exchange on Solana that enables users to deposit Liquid Staking Tokens (LSTs) to mint two synthetic assets: a stablecoin (hyUSD) and a leveraged token (xSOL).
stability-pool	A liquidity pool where users deposit `hyUSD` to earn yield while supporting the rebalancing of the exchange's collateral ratio.

03 — Findings

Overall, we reported 7 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	2
HIGH	0
MEDIUM	1
LOW	2
INFO	2

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-HYL-ADV-00	CRITICAL	RESOLVED	`LstRegistry::load` lacks strict validation of `remaining_accounts`, allowing potential manipulation by omitting or duplicating `LstBlocks`, artificially altering the collateral ratio.
OS-HYL-ADV-01	CRITICAL	RESOLVED	The protocol utilizes EMA prices, which lag behind real-time market movements. During sharp price swings, users may exploit this delay to mint under-collateralized stablecoins or levercoins, enabling arbitrage opportunities at the protocol's expense.
OS-HYL-ADV-02	MEDIUM	RESOLVED	An attacker could exploit the protocol's stability mode transitions to mint `xSOL` and `hyUSD` at the most favorevable fee setting.
OS-HYL-ADV-03	LOW	RESOLVED	The current Pyth oracle logic does not adequately handle clock drift, high price uncertainty, oracle freshness, or proper oracle account verification, resulting in inaccurate and unreliable price updates.
OS-HYL-ADV-04	LOW	RESOLVED	Zero-value operations, such as swapping, minting, transferring, and redeeming zero-value amounts, do not serve any purpose and may affect the correctness of these processes.

Collateral Ratio Manipulation CRITICAL OS-HYL-ADV-00

Description

`LstRegistry::load` fails to sufficiently validate the `remaining_accounts` array, which may result in collateral ratio manipulation. Since the function does not validate that the exact expected set of LSTs is provided, an attacker can omit specific `LstRegistryBlocks` from `remaining_accounts`. This will reduce the total collateral considered, artificially decreasing the collateral ratio, which may facilitate under-collateralized actions or faulty liquidation thresholds.

>_ hylo-exchange/src/state/lst_registry.rs RUST

```rust
pub fn load(
    remaining_accounts: &'info [AccountInfo<'info>],
) -> Result<LstRegistry<'info>> {
    // Split accounts into preamble and LST specific blocks
    let (calculator_accounts, lst_blocks) =
      remaining_accounts.split_at(SANCTUM_CALCULATOR_ACCOUNTS.len());

    // Load calculator accounts as SanctumContext
    let (spl, sanctum_spl, sanctum_spl_multi, marinade) = calculator_accounts
      .iter()
      .tuples::<(_, _, _, _)>()
      .map(SanctumContext::from)
      .collect_tuple()
      .filter(|(a, b, c, d)| {
        a.valid_as::<SplSolValCalc>()
          && b.valid_as::<SanctumSplSolValCalc>()
          && c.valid_as::<SanctumSplMultiSolValCalc>()
          && d.valid_as::<MarinadeSolValCalc>()
      })
      .ok_or(LstRegistryPreamble)?;

    // Load LST blocks
    let registry = lst_blocks
      .iter()
      .tuples::<(_, _, _, _)>()
      .map(TryInto::<LstRegistryBlock>::try_into)
      .collect::<Result<_>>()
      .and_then(|v| NonEmpty::from_vec(v).ok_or(LstRegistryEmpty.into()))?;

    [...]
}
```

Furthermore, there is no verification to prevent duplicates in `LstRegistryBlock`. Thus, it is possible to pass the same `LstBlock` multiple times, inflating the collateral ratio by counting the same assets repeatedly and allowing the attacker to over-leverage or avoid liquidation. Additionally, the `HarvestYield` instruction currently accepts an unchecked `lst_registry` account without verifying that it matches

the trusted registry stored in `hylo.lst_registry`.

Remediation

Ensure the `remaining_accounts` length matches the expected total of calculator accounts and LST blocks, and prevent duplicates in `LstRegistryBlock`. Additionally, add a verification to check that `lst_registry` equals `hylo.lst_registry`.

Patch

Resolved in PR #134.

Arbitrage Vulnerability Due to Imbalance in EMA Price CRITICAL OS-HYL-ADV-01

Description

Hylo Exchange relies on Pyth's exponentially-weighted moving average (EMA) price to value collateral during minting, instead of the more responsive spot price. Since EMA lags behind market movements, it creates an arbitrage opportunity at the expense of the protocol.

For example, the EMA price may remain artificially high during price drops, allowing attackers to buy assets at the lower spot price and mint stablecoins (`hyUSD`) utilizing the outdated, higher EMA price. To be clear, exploitation is only possible if there is at least a 1% spread between the spot price and the EMA price, sufficient to offset fees.

Proof of Concept

The vulnerability is illustrated in the below example where the `SOL` / `USD` price drops and under-collateralized stablecoins are minted:

1. As the `SOL` / `USD` price drops, `ema_price` becomes greater than `spot_price` . Assuming `ema_price` of $148, `ema_conf` of zero and a `spot_price` of $146, this reflects an average that has not yet fully adjusted to the recent drop in the spot price.
2. In `ExchangeContext` , `sol_usd_price` is set as `PriceRange { lower: ema_price - ema_conf; upper: ema_price + ema_conf }` , which, in this case, translates to `PriceRange {148, 148}` .
3. The user now buys `SOL` at the spot price. Consequently, the user mints `hyUSD` with the `SOL` they just bought at `spot_price` ($146), while the protocol values it at `PriceRange.upper` ($148) in `ExchangeContext.mint_conversion` .
4. Thus, this enables the user to mint `hyUSD` worth $148 utilizing only $146 worth of `SOL` .

Remediation

Utilize the spot price instead of the EMA price.

Patch

Resolved in PR #200.

Incorrect Fee Application MEDIUM OS-HYL-ADV-02

Description

Minting fees for both `xSOL` and `hyUSD` are determined based on the current `StabilityMode`. However, if a mint operation causes a transition to a less favorable mode (e.g., from `StabilityMode1` to `StabilityMode2`), the protocol still calculates the fee using the pre-transition mode.

Proof of Concept

The following sequence demonstrates how an attacker could leverage this to mint large amounts of `xSOL` while avoiding minting fees:

1. Suppose that the protocol is initially in `StabilityMode1`. The attacker observes that the system is close to the threshold for entering `StabilityMode2`.
2. The attacker submits two minting instructions back-to-back:
   - **Step 1:** Mint a small amount of `hyUSD`, reducing the collateral ratio just enough to trigger a transition to `StabilityMode2`.
   - **Step 2:** Immediately mint a large amount of `xSOL`, which now benefits from `StabilityMode2`'s zero-fee configuration.
3. As a result, the attacker pays only minimal fees for the first mint (under `StabilityMode1`), while the second, larger mint incurs no fees at all due to the updated mode.

This behavior undermines the protocol's intended fee structure and incentivizes strategic manipulation of mode transitions.

Remediation

Fee calculations should be based on the resulting stability mode after the mint operation, not the mode at the beginning of the transaction.

Patch

Resolved in PR #145.

Inconsistencies in Pyth Oracle Validation Logic LOW OS-HYL-ADV-03

Description

There are multiple instances of improper validations in the Pyth oracle where the Pyth price update logic fails to safeguard the users and the protocol from excessively high or incorrect price updates, affecting the overall functioning of the protocol. One such instance occurs in the comparison of Pythnet's `publish_time` (`PriceUpdateV2.price_message.publish_time`) with Solana's on-chain `clock_time` to check if the price update is fresh within `pyth::validate_publish_time`.

However, because these timestamps originate from separate systems, they may experience clock drift. Specifically, if Solana's clock lags behind real-world time, a valid Pyth update may be rejected if Pythnets timestamp exceeds that of Solana's, resulting in a subtraction underflow when computing the `delta` (shown below).

>_ hylo-exchange/src/pyth.rs RUST

```rust
/// Ensures the oracle's publish time is within the inclusive range:
///   `[clock_time - oracle_interval, clock_time]`
fn validate_publish_time(
  publish_time: i64,
  oracle_interval: u64,
  clock_time: i64,
) -> Result<()> {
    [...]
    let delta = clock_time.checked_sub(publish_time).ok_or(TimeUnderflow)?;
    if delta <= oracle_interval {
      Ok(())
    } else {
      Err(OracleNoPriceData.into())
    }
}
```

Also, the protocol fails to validate if both `PriceUpdateV2.price_message.conf` and `PriceUpdateV2.price_message.ema_conf` remain within acceptable bounds. Furthermore, `PriceUpdateV2.verification_level` should be checked as a security best practice to ensure that the oracle account has been fully verified off-chain (`VerificationLevel::Full`).

Remediation

Utilize `saturating_sub` when calculating the `delta` in `pyth::validate_publish_time` to avoid panics. Additionally, validate `PriceUpdateV2.posted_slot` to assess oracle freshness more reliably using Solana's slot-based mechanism, and ensure that the oracle account has been fully verified. Lastly, update the logic so that the protocol aborts if the `conf` / `price` ratio exceeds a safe threshold to protect against Pyth price updates with high uncertainty.

Patch

Resolved in PR #200.

Zero Value Operations LOW OS-HYL-ADV-04

Description

In `MintLevercoin`, `RedeemLevercoin`, and `RedeemStablecoin`, ensure that `amount_lst_to_deposit`, `amount_to_redeem`, and `amount_stablecoin_to_redeem` are greater than zero, respectively. Similarly, when swapping in `handle_lever_to_stable` and `handle_stable_to_lever`, verify `amount_levercoin` and `amount_stablecoin` are greater than zero, respectively. Allowing zero-value minting and redeeming is unnecessary, wastes computational resources, and adds noise to event logs.

Also, before performing transfers in the following functions, check that `fees_extracted.bits` is greater than zero to prevent minting or transferring zero-value fees into the `fee_vault`:

1. `MintLevercoin::deposit_collateral`.
2. `MintStablecoin::deposit_collateral`.
3. `RedeemLevercoin::redeem_collateral`.
4. `RedeemStablecoin::redeem_collateral`.
5. `handle_lever_to_stable` and `handle_stable_to_lever` in `Swap`.

Remediation

Add checks to restrict swapping, minting, transferring, and redeeming zero-value amounts.

Patch

Resolved in PR #155.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-HYL-SUG-00	Recommendation to introduce specific custom errors to enhance clarity and robustness in the `RedeemLevercoin` instruction.
OS-HYL-SUG-01	Recommendation to include slippage checks in `redeem`, `mint`, and `swap` handlers.

Improved Error Handling OS-HYL-SUG-00

Description

Introduce a custom error in `RedeemLevercoin::redeem_collateral` to explicitly handle the scenario when the `lst_vault` does not hold enough LST to fulfill the redemption (`amount_lst_out`). Also, when the system is in `Depeg` stability mode, attempting to redeem levercoin currently triggers a `NoValidLevercoinRedeemFee` error in `FeeController::redeem_fee`. However, this error may not accurately convey that the redemption is blocked due to the `Depeg` mode.

>_ *hylo-exchange/src/state/fee_controller.rs* RUST

```rust
fn redeem_fee(&self, mode: StabilityMode) -> Result<UFix64<N4>> {
    match mode {
        Normal => Ok(self.base_fee()),
        Mode1 => self
          .base_fee()
          .checked_add(&self.stability_fee_1())
          .ok_or(FeeArithmetic.into()),
        Mode2 => self
          .base_fee()
          .checked_add(&self.stability_fee_2())
          .ok_or(FeeArithmetic.into()),
        Depeg => Err(NoValidLevercoinRedeemFee.into()),
    }
}
```

Thus, it would be appropriate to implement a custom error, such as `RedeemNotAllowedDuringDepeg`, to provide a clear and accurate explanation of why the redemption is restricted.

Remediation

Implement the custom errors suggested above for better error handling and clarity.

Patch

Resolved in 24d4003.

Lack of Slippage Checks

OS-HYL-SUG-01

Description

Add slippage checks in `redeem`, `mint`, and `swap` handlers to protect users from front-running (sandwich) attacks and sudden price swings due to market volatility.

Patch

Resolved in PR #206.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.